

PENJELASAN FLOW KERJA TASK MANAGEMENT

Panduan Naratif Alur Kerja, Status Lifecycle & Proteksi Keamanan System - Next.js

1. Pendahuluan & Gambaran Umum Sistem

Sistem Task Management pada aplikasi portofolio Next.js ini dirancang menggunakan arsitektur modern berbasis App Router, TypeScript, dan Prisma ORM dengan basis data MySQL. Sistem ini bertujuan untuk mengelola seluruh tahapan pengerjaan tugas secara terstruktur, transparan, dan aman melalui mekanisme persetujuan berjenjang (dual-approval workflow).

Dalam pengembangan aplikasi ini, setiap tugas atau task memiliki siklus hidup yang terdefinisi dengan ketat pada skema database. Pengaturan hak akses dan validasi dilakukan secara mutlak pada sisi server untuk memastikan integritas data dan mencegah manipulasi oleh pihak yang tidak memiliki otorisasi.

2. Penjelasan 5 Status Lifecycle & Konsep Task Locking

Sistem mengenal 5 status resmi task yang disimpan di skema database Prisma, yaitu:

BACKLOG, OPEN, IN_PROGRESS, DONE, dan CLOSED.

Penting untuk dipahami bahwa LOCKED bukan merupakan status task tersendiri, melainkan sebuah kondisi proteksi keamanan (`isLocked === true`) yang aktif secara otomatis apabila sebuah task telah menyelesaikan tahapan persetujuan dan berada pada status DONE atau CLOSED.

- **Status BACKLOG:** Tahap awal saat tugas baru saja dibuat dalam perencanaan. Task masih belum ditugaskan secara aktif dan dapat dipindahkan ke status OPEN atau IN_PROGRESS.

- **Status OPEN:** Tugas telah disetujui untuk dikerjakan dan siap diambil atau dialokasikan kepada anggota tim (assignee). Task pada tahap ini masih bersifat editable.

- **Status IN_PROGRESS:** Tugas sedang dikerjakan secara aktif oleh anggota tim yang ditunjuk. Pada status ini, assignee dapat mengajukan permohonan persetujuan penyelesaian task (Request to Done) maupun permohonan penutupan task (Request to Close).

- **Status DONE:** Tugas telah selesai dikerjakan dan berhasil disetujui oleh Reviewer atau Project Owner via workflow Request to Done. Task dalam status DONE langsung terkunci (`isLocked = true`) dan menjadi Read-Only.

- **Status CLOSED:** Tahap akhir di mana tugas ditutup secara permanen via workflow Request to Close dengan persetujuan Project Owner. Task ini terkunci penuh dan tidak dapat diubah kembali.

PENJELASAN DETIL DUAL-APPROVAL WORKFLOWS

3. Alur Kerja Dual-Approval: Request to Done & Request to Close

Sistem ini menerapkan dua alur persetujuan terpisah untuk memastikan bahwa tidak ada task yang dapat diselesaikan atau ditutup tanpa melalui pengawasan dan verifikasi resmi:

A. Alur Kerja Request to Done (Menyelesaikan Pekerjaan Task)

Ketika seorang anggota tim yang bertugas (assignee) telah menyelesaikan seluruh kriteria pengerjaan task, dia tidak dapat secara sepihak mengubah status task menjadi DONE. Sebaliknya, anggota tim akan mengirimkan permintaan persetujuan melalui endpoint `/api/tasks/[id]/request-done`.

Tindakan ini akan mengeset atribut `doneRequestStatus` menjadi PENDING. Selanjutnya, Project Owner atau Reviewer yang memiliki otoritas akan memeriksa hasil pekerjaan:

- 1. Apabila Disetujui (Approve):** Status task berubah menjadi DONE, atribut `doneRequestStatus` berubah menjadi APPROVED, dan `isLocked` otomatis di-set true sehingga task menjadi Read-Only.
- 2. Apabila Ditolak (Reject):** Status task dikembalikan ke IN_PROGRESS, `doneRequestStatus` menjadi REJECTED, dan anggota tim harus melakukan perbaikan sesuai catatan penolakan.

B. Alur Kerja Request to Close (Menutup Task secara Permanen)

Request to Close merupakan alur yang berbeda dari Request to Done. Pengajuan penutupan task dapat dilakukan baik dari status IN_PROGRESS maupun status DONE melalui endpoint API `/api/tasks/[id]/request-close`, yang mengubah `closeRequestStatus` menjadi PENDING.

Berbeda dari Request to Done, persetujuan penutupan task HANYA boleh dilakukan oleh Project Owner secara langsung. Hasil dari alur persetujuan ini adalah:

- 1. Apabila Owner Menyetujui (Approve):** Status task berubah secara mutlak menjadi CLOSED, `closeReviewedById` diisi ID Owner, `closeReviewedAt` mencatat tanggal approval, dan `isLocked` bernilai true.
- 2. Apabila Owner Menolak (Reject):** Status task dikembalikan persis ke status valid sebelum pengajuan (yaitu IN_PROGRESS atau DONE) dan `closeRequestStatus` menjadi REJECTED.

ATURAN KETAT NOTIFIKASI EMAIL REQUEST TO CLOSE:

Saat seorang anggota mengajukan Request to Close, sistem akan memicu layanan pengiriman email notifikasi (`sendTaskCloseNotification`). Email notifikasi ini WAJIB HANYA dikirimkan kepada **Project Owner (task.project.owner.email)**.

Sistem melarang keras pengiriman email kepada requester, creator, assignee lain, atau admin lain guna menjaga kerahasiaan komunikasi dan pengawasan penuh di tangan Project Owner.

4. Penjelasan Otorisasi Berlapis & Security Boundary Server

Dalam arsitektur sistem ini, keamanan dan otorisasi sepenuhnya dipaksakan pada sisi Server (Backend). Sesuai dengan prinsip dasar keamanan aplikasi, pemeriksaan otorisasi pada Frontend UI (seperti menyembunyikan tombol Edit atau mentautkan status disabled) HANYA berfungsi untuk meningkatkan kenyamanan pengguna (UX) dan BUKAN sebagai benteng keamanan.

Setiap permintaan mutasi data pada API Endpoint harus melewati 6 lapisan verifikasi berurutan:

- 1. Authentication Session:** Memastikan identitas pengguna terverifikasi via NextAuth session.
- 2. Workspace & Project Access:** Memastikan pengguna merupakan anggota aktif dari project terkait.
- 3. Task Object Access:** Memastikan task memang berada di bawah project milik pengguna.
- 4. Role & Workflow Permission:** Memvalidasi matriks izin peran (Owner, Admin, Member).
- 5. Lock State Verification:** Memeriksa apakah `isLocked === true` atau status `DONE/CLOSED`.

MEKANISME PROTEKSI TASK LOCKING (READ-ONLY GUARD):

Apabila sebuah task telah berada pada status `DONE` atau `CLOSED`, atau atribut `isLocked` bernilai `true`, maka seluruh permintaan pengubahan data (Edit title, description, priority, assignee,) Tj ET BT /F2 9 Tf 0.20 0.25 0.33 rg 45 454 Td (

HTTP 403 Forbidden via helper `getTaskLockedResponse()`.

5. Penomoran Otomatis, Audit Trail & Zero Layout Shift

- Penomoran Task Otomatis (Auto-Numbering):

Setiap tugas baru diberikan nomor unik berurutan secara otomatis melalui transaksi database atomic dengan helper `generateNextTaskNumber(tx)`. Format penomoran menggunakan standar **TSK-000001, TSK-000002, dan seterusnya.**

- Audit Activity Trail (Pencatatan Aktivitas):

Setiap perubahan status, pengajuan approval, penolakan, maupun mutasi field diwajibkan mencatat riwayat aktivitas secara otomatis via `logTaskActivity()`. Log ini mencatat siapa yang melakukan perubahan, kapan dilakukan, serta nilai sebelum dan sesudah perubahan.

- Standar UI UX Zero Layout Shift:

Pada komponen tabel task frontend, pengambilan data saat pagination atau filtering memisahkan state `isLoading` (initial fetch) dan `isFetching` (background fetch). Tabel tidak boleh unmount atau menghilang (flicker), menjaga kenyamanan navigasi pengguna tanpa layout shift.